

Railway Gauges Historical Network Analysis

End-to-end data ingestion, track growth trajectories, gauge standardisation modeling, and multi-variable analytics.

Dataset: railway_gauges.csv (49 Records)

Span: 1964–65 to 2012–13

Tech: Python, Pandas, NumPy, Matplotlib

1. Scenario Breakdown & Quantitative Findings

Scenario 1: Basic Data Loading & Cleaning

Loaded dataset (49 rows × 5 columns: Year, Broad Gauge, Metre Gauge, Narrow Gauge, Total). Replaced nulls with 0 and cast gauge columns to numeric data types.

Scenario 2: Total Railway Track Growth Trend

Extracted Year and Total attributes. Network grew from 6,876 (1964–65) to 7,172 tracks (2012–13) (+4.30%). Generated plot: total_tracks_growth.png.

Scenario 3: Post-2000 Expansion & Dominance Analysis

Filtered years ≥ 2000 . Grouped bar chart verified that **Broad Gauge heavily dominates recent years** (>90% share), growing to 6,577 tracks while Metre Gauge fell to 379. Generated plot: post_2000_gauges_bar.png.

Scenario 4: Feature Engineering & Cumulative Contribution

Computed cumulative sum: Broad Gauge = 224,066 (65.0%), Metre Gauge = 101,798 (29.5%), Narrow Gauge = 18,953 (5.5%). Pie chart: gauge_contribution_pie.png.

Scenario 5: Advanced NumPy Growth Modeling & Unigauge Shift

Created percentage columns. Applied `np.diff()` to isolate annual growth rate; peak single-year net network growth occurred in **2003–04 (+125 tracks)**. Confirmed unigauge standardization into Broad Gauge.

2. Visual Representation

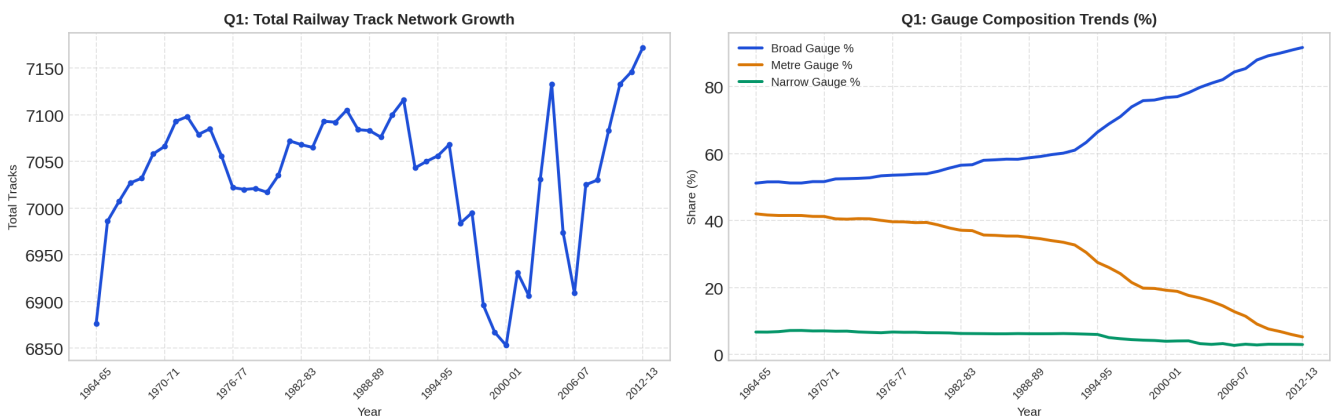


Figure 1.1: Total Network Growth Trajectory (Left) & Historical Gauge Composition Share (%) (Right).

3. Python Code Snippet

```
import os, pandas as pd, numpy as np, matplotlib.pyplot as plt
csv_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "railway_gauges.csv")
df = pd.read_csv(csv_path).fillna(0)
for c in ['Broad Gauge', 'Metre Gauge', 'Narrow Gauge', 'Total']:
    df[c] = pd.to_numeric(df[c], errors='coerce').fillna(0)
growth = np.diff(df['Total'].to_numpy())
df['% Broad Gauge'] = (df['Broad Gauge'] / df['Total']) * 100
print(f"Peak Total Network: {df['Total'].max()} in FY {df.loc[df['Total'].idxmax(), 'Year']}")
```